

TRAINEE LAB GUIDE

Shopify Agentic AI Developer Bootcamp — Koenig Solutions for V Group Inc.

Lab 2.2

A Calculator/Lookup Agent (Tool Use)

Module	2 — Building Agents with the Claude API & Agent SDK
Topic	2.2 — Tool Use / Function Calling
Estimated time	30–40 minutes
Tools needed	Terminal, Node.js 20+, a code editor, an active Claude API key
Prerequisite	Lab 2.1 complete; Topic 2.2 briefing (tool schemas, the tool-call round trip) on the program website

What You're Building

GOAL

A Node.js agent with one custom tool — a four-function **calculator** (add/subtract/multiply/divide) — that wires the full tool-call round trip end to end: Claude decides whether a tool is needed, your code runs the real calculation, and the result gets sent back so Claude can answer using it.

This lab uses the calculator variant of the exercise (rather than a product lookup table) because it needs no external data file — everything the agent needs is generated on the fly, in your own code.

Steps 1–3: Set Up the Project

1 Create a project folder

In a terminal, outside of any other project:

TERMINAL

```
mkdir calculator-agent
cd calculator-agent
npm init -y
```

What you'll see: a new folder containing a `package.json` file.

2 Store your API key safely

Create a file named `.env` in the project folder with your Anthropic API key:

.ENV

```
ANTHROPIC_API_KEY=sk-ant-your-key-here
```

Before you go further: create a `.gitignore` file containing just the line `.env` — this key should never end up committed to git, even in a lab exercise.

3 Install the SDK and enable ESM

TERMINAL

```
npm install @anthropic-ai/sdk dotenv
```

Open `package.json` and add `"type": "module"` at the top level so you can use modern `import` syntax.

What you'll see: `@anthropic-ai/sdk` and `dotenv` listed under `dependencies` in `package.json`.

Steps 4–5: Write the Agent

4 Open Claude Code and describe the task

In your project folder, run `claude` to start Claude Code, then give it this spec:

PROMPT TO CLAUDE CODE

```
Build a Node.js script (ESM, call it calculator-agent.js) that runs
an interactive terminal agent using the @anthropic-ai/sdk package:

1. Loads ANTHROPIC_API_KEY from a .env file via dotenv
2. Defines exactly one tool called "calculate" with this input_schema:
  { operation: "add"|"subtract"|"multiply"|"divide" (enum),
    a: number, b: number } – all three fields required
3. Reads a line of input from the terminal (use Node's readline),
  sends the full conversation history plus the tool to the Messages
  API (model: "claude-sonnet-5", max_tokens: 1024, tool_choice: "auto")
4. If response.stop_reason is "tool_use", actually PERFORM the
  requested operation in JavaScript – do not let Claude invent the
  result – then send it back as a tool_result content block in a
  new user turn, and call the API again for Claude's final answer
5. If the operation is "divide" and b is 0, return a tool_result with
  is_error: true and a short message instead of throwing or crashing
6. Print Claude's final text reply, then loop back and prompt for the
  next line, keeping full conversation history across turns

Do not add a product-lookup tool – one calculate tool only.
```

5 Review the diff before accepting

Claude Code will propose creating `calculator-agent.js` and updating `package.json`. Before accepting, check:

Check	Why it matters
Does <code>input_schema</code> restrict <code>operation</code> to the four allowed values (an <code>enum</code>)?	Claude can't invent an operation your code doesn't handle
Does the script check <code>response.stop_reason === "tool_use"</code> before trying to execute anything?	A plain-text reply has no tool call to run
Does the actual arithmetic happen in your JS code, not get invented by Claude?	A calculator tool that doesn't calculate defeats the whole lab

Does dividing by zero return an `is_error` tool result instead of crashing the process?

One bad input shouldn't kill the whole REPL session

Does the `tool_result` get sent back in a new turn, with the API called again?

Claude needs the real result before it can give a final answer

If anything's missing, don't fix it yourself — ask Claude Code directly: "check `stop_reason` before reading the tool input" or "guard the divide operation against `b == 0` and return an error tool result." That request-and-iterate step is the point of the exercise.

Steps 6–7: Run and Verify

6 Install dependencies and try it

TERMINAL

```
npm install
node calculator-agent.js
```

Try these three lines, one at a time, to exercise all three behaviors:

You: What's the capital of France?

→ *no tool call — Claude answers directly from what it already knows*

You: What's 128 divided by 8?

→ *a real tool call with operation="divide", a=128, b=8; final answer reflects the actual result, 16*

You: What's 10 divided by 0?

→ *tool returns an error result; Claude explains it can't divide by zero — script keeps running*

What you'll see: the first question gets an immediate plain-text reply with no visible tool step; the second and third trigger a brief pause while the tool result round-trips before the final answer prints.

7 Confirm the acceptance checklist

- ☐ Asking a question the tool can't answer gets a plain-text reply, no tool call
- ☐ Asking something the tool *can* answer triggers a real tool call with correct arguments
- ☐ The tool's actual result (not a guess) appears in Claude's final answer
- ☐ An invalid input (e.g. divide by zero) is handled without crashing the script

Troubleshooting

Symptom	Likely cause	Try this
Claude answers a math question directly, without calling the tool	Tools array not attached on every call, or the tool description is too vague	Confirm <code>tools</code> is passed on every request in the loop, and the description clearly states what the tool is for

Script crashes when a tool call comes back	Missing check for <code>stop_reason</code> before reading tool input	Add an explicit <code>if (response.stop_reason === "tool_use")</code> branch
Dividing by zero crashes the script	No guard around the division itself	Ask Claude Code to check <code>b === 0</code> and return an <code>is_error</code> tool result instead of throwing
Claude's final answer doesn't match the real math	The <code>tool_result</code> content sent back doesn't match what your JS actually computed	Log the JS result right before sending it back and compare by hand
<code>ANTHROPIC_API_KEY</code> is undefined	<code>.env</code> not loaded, or in the wrong folder	Confirm <code>dotenv.config()</code> runs before the client is created

Where This Goes Next

This exact round trip — define a schema, dispatch on `stop_reason`, run the real logic yourself, send the result back — is what Module 3 reuses to wrap real Shopify Storefront and Admin API calls as tools instead of a calculator. Topic 2.3's Claude Agent SDK then takes over managing this same loop for you automatically.